

TESTE CONCEITUAL - TOTVS

Ofensores destacados:

- A vitrine consulta o ERP a cada acesso e fica lenta. O negócio precisa reduzir latência sem exibir preço ou disponibilidade incorretos.
- Clientes compram o mesmo item sem estoque. A solução precisa impedir overselling.
- A API do ERP demora para faturar o pedido. A jornada precisa tolerar timeout, retry e processamento assíncrono sem perder rastreabilidade.

PERGUNTA:

1. Para cada um dos 3 problemas identificados:
 - O que você acredita estar causando o problema na raiz, não apenas o sintoma?
 - Qual é o impacto para o cliente, para o negócio e para a operação?
 - Compare 2 ou 3 caminhos de solução, considerando custo, complexidade, latência, consistência e esforço operacional.

Inclua uma visão de arquitetura para 30 a 90 dias, citando cache, fila, banco próprio da loja, workers, sincronização, observabilidade e reconciliação com o ERP.

RESPOSTA:

O problema da lentidão na vitrine pode estar atrelado ao volume de dados e requisições, e a arquitetura do ERP. Esse problema de lentidão impacta principalmente na experiência do usuário, podendo trazer uma queda na conversão de vendas e sobrecargas no sistema. Podemos resolvê-la com algumas opções, como:

- Adicionar um cache (cache-aside) na consulta ao ERP. Essa seria a solução mais simples, barata, de baixo esforço e que reduziria mais a latência, visto que o cache trabalharia com dados em memória. Porém, considerando que consumimos os dados do ERP através de uma API, não conseguiríamos observar de forma simples quando um novo item foi adicionado, tornando a invalidação do cache mais difícil. Poderíamos seguir com essa solução através do dimensionamento de um TTL que fosse aderente a periodicidade de alteração do catálogo, mas ainda sim, poderiam haver inconsistências nos dados, já que ainda teria a possibilidade de o usuário receber um dado do cache que já foi alterado na base do ERP. Em uma segunda abordagem para invalidação do cache, poderíamos pensar em um outro serviço responsável por monitorar o banco de dados do ERP e comunicar o cache para descartar o dado registrado, porém isso começaria a aumentar complexidade, esforço e custo.
- Implantar um banco de dados próprio para a loja com uma estratégia de sincronização com o banco do ERP. O processo de sincronização poderia ser feito por um serviço intermediário em segundo plano, com uso de filas ou brokers, assim como soluções já existentes que fazem esse sincronismo, como alguns plugins do Kafka Connect. Essa solução implica em aumento de custo com instâncias próprias de bancos de dados, complexidade média para garantir o total funcionamento do serviço intermediário caso seja necessário implementá-lo manualmente, reduziria a latência, considerando que o banco local seria uma versão menor do que o banco do ERP, contendo apenas os dados necessários, teria uma consistência mais

controlada, porém ainda com a presença de uma inconsistência eventual até o serviço de sincronização executar, e teria um esforço operacional maior em relação a um cache simples.

- Implantar um banco de dados próprio para a loja trabalhando de forma isolada do ERP. Dessa forma, poderíamos exportar a base de produtos já cadastrados no ERP e importar no banco próprio da loja, assim teríamos também uma versão reduzida do banco de dados contendo apenas o necessário para a vitrine. Essa solução ainda tem um custo maior que a implantação de um cache, mas considerando que não teremos o uso de nenhum serviço extra, ela se torna uma opção mais econômica do que a anterior. Possui uma complexidade baixa, pois seria necessário apenas a implantação de um novo serviço de banco de dados, garantiria consistência, já que o cadastro de produtos seriam migrados para a plataforma da loja, reduziria latência, e teria baixo esforço. Contudo, o grande problema seria que os dados do catálogo de produtos ficariam obsoletos no ERP, e para solucionar isso, seria necessário retomar o aumento de custo e complexidade, trazendo uma solução com o fluxo inverso para a sincronização dos dados em relação à segunda sugestão de solução, onde a sincronização seria feita a partir do banco da loja para o ERP, tirando a consistência eventual da vitrine e movendo ela para o ERP, além de ser necessário a permissão de escrita no banco de dados do mesmo.

O problema de overselling ocorre por conta da concorrência das requisições. Vamos imaginar uma primeira requisição para a compra do produto A e, praticamente no mesmo tempo, uma segunda requisição para a compra do mesmo produto. A primeira requisição chega, é feita a validação da disponibilidade em estoque e antes de efetuar a compra e dar baixa, a segunda requisição chega e faz a mesma validação de disponibilidade. A primeira requisição termina e efetua a baixa no estoque, porém agora, a segunda requisição já avaliou o estoque em um estado anterior quando ainda havia disponibilidade, permitindo a segunda requisição também prosseguir com a compra, resultando na venda de um produto sem estoque. Esse tipo de problema pode trazer muita insatisfação de clientes, além de encargos a loja para estorno de um dos pedidos. Há diversas maneiras de solucionar esse tipo de problema:

- Reservar o estoque no início do checkout. Dessa forma, quando o cliente inicia o checkout, já separamos um item em estoque para ela. Esse checkout expira após um determinado tempo se a compra não for realizada, devolvendo ao estoque o produto reservado para aquele processo de compra. Se a compra for efetuada, a baixa no estoque pode ser computada completamente. Essa abordagem é uma opção barata, pois não há custos extras de infraestrutura, possui complexidade e latência baixa, garante consistência e com esforço operacional relativamente baixo para implementação.
- Realizar operações de checagem e compra de forma atômica com o banco de dados. Em bancos SQL, por exemplo, podemos aplicar uma query de UPDATE junto com um WHERE validando a disponibilidade do estoque, exemplo:

```
UPDATE products SET stock = stock - 1 WHERE id = {productId} AND stock >= 1;
```

Isso garante que a atualização ocorra apenas em produtos que possuam estoque disponível, contudo, em sistemas com réplicas ou shardings de bancos de dados, devemos nos atentar para que esse update vá sempre para a réplica principal, para evitar que

inconsistências eventuais entre as réplicas aplique o estoque de forma incorreta. Essa solução também é de baixo custo, complexidade, esforço e latência, além de garantir consistência. O problema dessa abordagem está em diferentes pontos da linha do tempo da compra, como o fato de que, se dois clientes finalizam praticamente ao mesmo tempo uma compra de um mesmo produto, um deles iria conseguir o produto e o outro receberia um erro por falta de estoque, isso após a operação de preenchimento de endereço, pagamento, entre outros, gerando uma péssima experiência aos usuários. Além disso, se houver falha em qualquer outra parte do processo, como no pagamento, esse UPDATE direto no banco de dados não possui um rollback automatizado, sendo assim, necessário atualizar a quantidade do estoque de volta para a anterior. Essa situação também se aplica a carrinhos com múltiplos produtos, pois seria necessário um UPDATE atômico por item, e mesmo assim, se o item A existe em estoque, mas o item B não, teríamos que reverter a baixa do item A novamente.

- Utilizar de locks nas operações, podendo ser pessimistic locks ou distributed locks. Dessa forma, garantimos que todo o fluxo seja cadenciado e dependente da disponibilidade dos produtos. É uma solução que traz aumento de complexidade, custo e esforço. Com o pessimistic lock, faríamos todo o fluxo de validação e baixa em estoque através de uma transaction de banco de dados, mas na necessidade de interação com outros serviços e dependências de outros processos, poderíamos utilizar um distributed lock com Redis. O problema aqui é que, essa operação se tornaria um gargalo em produtos de alta demanda devido a estar executando transações longas e travando os demais processos durante a execução.

Por fim, para o problema de lentidão no processamento dos pedidos. Ele também pode estar relacionado a arquitetura ou problemas de instabilidade no ERP, já que as chamadas são síncronas e mantêm o usuário aguardando o retorno com sua resposta. Isso pode ocasionar em uma experiência ruim para o usuário, e até mesmo, perda de pedidos, exigindo estorno ao cliente, ou reproprocessamento manual e atraso nas entregas. Podemos solucionar essa questão de algumas formas:

- Utilizar filas para o processamento assíncrono. Assim, quando o cliente finaliza a compra, a requisição faz a publicação do pedido em uma fila e devolve uma resposta imediata com status 202 (Accepted) para o usuário. Para garantias de processamento, teríamos implementações de retry com exponential backoff, chaves únicas, como orderId e/ou correlationId para garantir a unicidade da requisição (idempotência), e levaríamos em conta um sistema de timeout que aguarda o acknowledge da fila para garantia de entrega ao broker. Em caso de falha após um limite de tentativas no retry, a mensagem é enviada para uma Dead Letter Queue (DLQ) para análise manual, ou um posterior reproprocessamento. Junto com a mensagem perdida, é interessante manter logs estruturados com latência, tentativas e resultados, possibilitando a análise posterior da falha. Do outro lado, como consumidor dessa fila, teríamos workers dedicados para realizar o processamento assíncrono e atualizar o status do pedido no banco de dados. Contudo, essa solução abre brechas para o problema da escrita dupla (dual-write problem), onde no momento em que o cliente faz a requisição, o sistema pode falhar e trazer cenários como a escrita do pedido no banco de dados, mas não a publicação da mensagem na fila, ou o inverso, dependendo da sequência lógica no código, assim, o pedido nunca seria processado. Para solucionar esse problema, podemos utilizar o Transactional Outbox Pattern, que consiste em criar uma tabela intermediária onde

serão publicadas as mensagens que devem ser enviadas para processamento através da fila, e além disso, um worker intermediário que fará um polling nessa tabela e publicará na fila. Essa solução traz complexidade, esforço e custos elevados, baixa latência para o usuário com a resposta imediata com status 202, e consistência nos dados, porém, com um pequeno atraso devido ao processamento de eventos assíncronos.

- Aplicar um polling simples com workers dedicados na tabela de pedidos comparando seus status. Dessa forma, seria necessário apenas registrar o pedido no banco de dados e o worker seria responsável por enviá-lo para o ERP, aguardar o processamento e atualizá-lo no banco de dados. Ainda teriam que ser tratados questões como idempotência e um possível circuit breaker para momentos em que o ERP estivesse indisponível. É uma solução de baixo custo, complexidade, latência e esforço, e garante consistência de dados. Contudo, considerando o fluxo de milhões de usuários, essa opção pode começar a se tornar um gargalo, a menos que haja replicação dos workers de processamento, fazendo com que o serviço seja mais “elástico”, ou seja, podendo trabalhar com muitos workers em alguns momentos, e poucos em outros, o que pode causar imprevisibilidade e aumento na variabilidade dos custos, tornando ele diretamente proporcional ao volume de acessos.

Para o plano de 30 á 90 dias, eu faria de forma faseada:

1º Fase - Apagar os incêndios (entregue entre os dias 7 e 15):

Na fase inicial, temos muitos problemas críticos acontecendo ao mesmo tempo e não podemos esperar 30 dias para começar a entregar soluções.

Começaria com um cache simples para a vitrine, entendendo a existência da inconsistência entre o catálogo do ERP e a vitrine, implementaria a sessão reservada no processo de checkout para evitar o overselling e aplicaria de forma temporária o polling simples para processamento assíncrono dos pedidos. Além disso, implementaria uma primeira camada de logs estruturados que ajudasse a entender os problemas pelos quais estamos passando na aplicação.

2º Fase - Reduzir custos e otimizar processo (entregue entre os dias 40 e 60):

Com o caos mitigado, iria para a redução de custo e otimização para a solução do problema de lentidão nos pagamentos. Para determinar essa sequência de resolução, levei em consideração que a inconsistência no catálogo da vitrine e o ERP causada pela adição do cache na fase anterior, não seria um grande impacto ao usuário, considerando que para lojas que não trabalham como marketplaces ou revendedoras, catálogos tendem a ser mais estáticos. Dessa forma, traria a implementação da solução de processamento assíncrono com filas, Outbox Pattern, DLQs e as demais estratégias mencionadas na sugestão 1 acima de solução para este problema. Isso traria uma maior confiabilidade e resiliência no processo, além de trazer os custos para um patamar mais previsível, sem a necessidade de tanta elasticidade nos serviços quanto antes, e um monitoramento mais refinado. Aqui, poderíamos aproveitar para metrificar percentuais de cache miss, tempo na DLQ, taxa de retry, volume de dados nas filas, etc.

3º Fase - Resolver inconsistências (entregue entre os dias 70 e 80):

Com um processo mais resiliente na compra, buscaria solucionar a inconsistência de dados na vitrine causada pelo cache da Fase 1. Para isso, iria para a solução 2, com a

criação de um banco de dados local para a loja que é sincronizado de forma assíncrona com os dados no ERP. Dessa forma, a inconsistência passaria de alguns minutos, para alguns segundos, podendo ser até imperceptível ao usuário. Manteria o cache, mas agora com uma estratégia de invalidação e com TTLs mais curtos, conseguindo realizar o gerenciamento do mesmo a partir do serviço de sincronismo do banco de dados. Além disso, seria importante construir um monitoramento da base do ERP para atuar de forma periódica, como um certo nível de redundância, garantindo que se a sincronização perder algum dado, essa sincronização periódica colete ele.

4º Fase - Refinamento (entregue até o dia 90):

Toda solução sempre pode ser melhor de alguma forma. Esses últimos 10 dias seriam dedicados a refinamentos menores, melhorias na observabilidade do sistema, entre outros.

PERGUNTA:

2. Sobre a vitrine de produtos, preços e disponibilidade:

- Onde colocaria cache? Explique o papel de cada camada.
- Descreva TTL, invalidação, cache-aside/refresh-ahead, fallback, prevenção de cache stampede.
- Descreva métricas para validar ganho sem gerar dados obsoletos:
 - Se o cache realmente melhorou performance/custo.
 - Se essa melhora não veio às custas de entregar informação velha ou incorreta.

RESPOSTA:

Adicionaria um cache local na aplicação, que responde com latência praticamente zero e serve como uma primeira linha para filtrar as solicitações ao banco de dados, e também funcionaria como um fallback para a próxima camada de cache, que seria um cache-aside com Redis, que por funcionar de forma centralizada, garante consistência entre as instâncias, mas por ser um serviço externo, tem um pequeno aumento na latência, e além disso, comporta um volume maior de dados. Poderíamos usar um TTL em torno de 1 hora, com uma estratégia de adicionar um Jitter (mais ou menos 2 minutos) para evitar que todas as chaves expirem juntas, porém com um mecanismo de invalidação vinculado ao serviço de sincronização de bases entre o ERP e o banco de dados da loja, dessa forma, sempre que uma alteração ocorrer em algum produto, o serviço de sincronização invalida o cache obsoleto e registra um novo. Quando o TTL estiver próximo de expirar, para evitar o cache stampede, podemos também utilizar a estratégia de refresh-ahead para antecipar a renovação do cache, contudo, devemos ter a consciência que isso aumenta a complexidade, pois exige a implementação de um serviço de scheduling para fazer essa renovação de forma assíncrona, além de termos o custo computacional de recalcular o cache para possíveis produtos que não são mais vendidos (ou vendidos com baixa frequência). Há também a possibilidade de cache eviction, caso o limite do cache seja atingido, e para isso, podemos seguir com uma estratégia de LFU (Least Frequently Used), assim priorizamos os produtos mais frequentemente consultados pelos clientes. Para tratarmos casos de falha ou queda de ambos os sistemas de caching, podemos tomar uma decisão de negócio e aceitar servir o último valor visto em cache, mesmo ele podendo ser um valor antigo, ou como um fallback, fazer um bypass das requisições mandando elas direto ao nosso banco de dados. Para metrificar isso, podemos levantar índices de cache

miss e cache hits, taxas de divergência entre o cache e o ERP, tempo médio entre a mudança no ERP e a invalidação do cache, ou contagem de vezes que um dado servido do cache divergiu do valor real detectado no sincronismo.

PERGUNTA:

3. A liderança quer detectar degradação e furos de estoque antes de reclamações de clientes. Sem depender de conta real em ferramenta:
 - Que logs estruturados você instrumentaria? Quais campos seriam obrigatórios?
 - Que métricas criaria usando counters, gauges e histograms para cache, checkout, fila e ERP?
 - Que traces distribuídos seriam essenciais nos fluxos GET /products e POST /checkout assíncrono?
 - Que SLI/SLO, alertas, dashboard e runbook configurariam no Datadog ou equivalente?

RESPOSTA:

Para criar um log estruturado para essa solução, manteria como obrigatório campos como:

- orderId: Id da compra realizada;
- correlationId: Id único para o fluxo da requisição;
- productId: Id do produto comprado;
- requestedAmount: Quantidade solicitada para o produto;
- availableAmount: Quantidade disponível para venda;
- service: Serviço responsável por disparar o log;
- event: O evento que identifica a ação capturada pelo log;
- result: O resultado produzido pela ação.

Dessa forma, poderíamos aplicar diferentes tipos de logs durante o processo de compra, sendo alguns deles:

- Para antecipar o problema, poderíamos disparar um log na sincronização entre a base da loja e do ERP, em caso de divergências encontradas, mostrando a quantidade que cada base possui. Dessa forma, poderíamos analisar se é uma falha sistêmica ou operacional, e trabalhar nos processos de mitigação;
- Na etapa inicial da compra, onde é realizado a checagem do estoque, disparamos um event stock_check, que pode produzir um result success ou rejected_insufficient_stock (por exemplo). Com essa informação, poderíamos acompanhar o número de pedidos que falharam por falta de estoque;
- Como alerta, podemos adicionar um log que é disparado quando o estoque fica negativo. Isso não antecipa o problema, mas permite identificar com mais prontidão a incidência.

Para monitoramento de serviços, podemos aplicar counters em métricas de cache, como cache miss e cache hits, para avaliar a assertividade do sistema de caching. Podemos analisar número de pedidos realizados divididos por status para o checkout, permitindo um acompanhamento em tempo real da quantidade de pedidos processados e quantidade de falhas. Para filas, contagens de retry e o número de mensagens em DLQs

nos dá uma imagem de como os serviços que enviam e recebem as mensagens estão, pois se está ocorrendo muitas tentativas, isso pode levantar um questionamento sobre a eficiência e disponibilidade dessa parte do sistema.

Podemos utilizar também gauges para medir recursos e volume de dados em cada serviço ou componente do sistema, por exemplo volume ocupado do cache, para prever cache eviction, volume de mensagens por segundo nas filas, e recursos computacionais como RAM e CPU das APIs internas e do ERP. Além disso, também podemos fazer o uso de histogramas para análise de latência de requisições, separando métricas como p90, p95 e p99, e também separarmos a quantidade de falhas por tipo, permitindo criar um tipo de Diagrama de Pareto com as principais falhas.

Seria essencial a aplicação de tracing distribuído na etapa de consulta de dados para o fluxo GET /products, revelando assim, quanto tempo é gasto com o dado vindo do cache e, em caso de cache miss, qual o tempo gasto com a consulta ao banco de dados. Seria de extrema importância também, monitorarmos o fluxo de sincronização da base com o ERP que ocorre de forma assíncrona e o fluxo de refresh-ahead de renovação do cache, assim podemos metrificar a eficiência em milissegundos destes serviços. Já para o fluxo de POST /checkout, podemos metrificar todo o processamento da compra, visualizando métricas do tempo gasto com o salvamento do pedido e do evento no banco de dados, a publicação na fila, a coleta da mensagem pelo consumer, o processamento pelo ERP, até a devolução da resposta.

Poderíamos definir as seguintes opções abaixo como métricas interessantes de SLI e SLO:

- Para validar a saúde do cache na vitrine podemos utilizar uma métrica de cache hits com um percentual acima de 90%, e além disso, manter 99% das requisições na vitrine respondendo em um tempo menor do que 200 ms.
- Para constatar que o problema de vendas sem estoque não está ocorrendo mais, seria interessante também termos uma métrica sobre quantas vezes o estoque ficou negativo e a tolerância para isso deve ser zero.
- Para validar que no fluxo de compra não há mais lentidão, podemos adicionar a medida de 99% das requisições de compra serem resolvidas em menos de 500 ms. Podemos também adicionar uma métrica para o percentual de mensagens que chegam nas DLQs, estipulando um máximo de 0,1%.

Para a montagem de um dashboard e um sistema de alertas, podemos visualizar:

- a latência da vitrine de produtos com as métricas p50, p95 e p99, adicionando alertas para um p95 maior que 200 ms e um p99 maior que 1 segundo;
- o percentual de cache miss, alertando quando abaixo de 80%, que pode sinalizar um TTL mal programado ou a ocorrência de um cache stampede;
- o volume de pedidos por status;
- a profundidade das filas e das DQLs, onde se passar 0.1% em 30 minutos deve disparar um warning;
- a contagem de problemas em estoque nas últimas 24 horas, que deve disparar um alerta crítico caso ocorra;
- problemas de sincronização entre o banco local e o ERP, também alertando para revisão de estoque.

Para casos de valores de estoques negativos, representando falhas no sistema de checkout, podemos ter um runbook de severidade crítica e alerta imediato para o time seguindo o seguinte cronograma para diagnóstico:

1. Precisamos identificar o ID do produto afetado e quantos itens além do permitido foram vendidos.
2. Consultar os logs relacionados a aquele produto para obter mais informações sobre cada pedido realizado naquele intervalo.
3. Podemos verificar logs com o stock check para validar se houve concorrência e diagnosticar se foi algum problema nesse mecanismo, ou se foi algo externo a esse fluxo, como um bug ou uma escrita manual no banco de dados.
4. Podemos validar também os logs de divergência de estoque entre banco local e ERP, para validar se a quantidade veio distorcida do ERP ou se já estava no banco local da loja.

Junto com o diagnóstico, podemos direcionar parte do time para a mitigação imediata do problema. Sendo assim, devemos bloquear as novas compras deste produto que foi afetado, evitando que a quantidade negativa no estoque aumente. Em seguida, precisamos identificar os pedidos afetados e acionar o time de operação para reposição de estoque ou, se não for possível, o cancelamento da compra e o estorno, antes mesmo da reclamação do cliente.

Quando o estoque for normalizado, reabilitamos a venda daquele produto, documentamos a causa raiz identificada no diagnóstico e, se houve uma lacuna no mecanismo do sistema ou nos testes automatizados, criamos um item de ação para correção deste problema e intensificação dos testes na esteira de deploy.

PERGUNTA:

4. Sobre o furo de estoque no checkout:
 - Explique por que uma checagem simples de estoque é insuficiente. Compare atomic update condicional, pessimistic lock, reserva de estoque e distributed lock.
 - Como usaria idempotência para tolerar retry, duplo clique e reprocessamento? Como testaria que a solução evita overselling?

RESPOSTA:

Uma checagem simples de estoque não é suficiente pois, com o aumento de usuários na plataforma, existe a possibilidade de requisições concorrentes consultarem o mesmo produto em estoque antes que a outra requisição consiga efetuar a saída do mesmo. Para trabalhar uma solução, temos a opção do Atomic Update, que consiste em realizar operações atômicas no banco de dados, como a conferência do estoque junto com a atualização dele, em uma única query (UPDATE com WHERE). Essa abordagem é simples porém não garantida em alguns cenários, como em carrinhos com múltiplos produtos, onde um produto existe em estoque, mas o segundo não, sendo necessário reverter manualmente a baixa realizada no primeiro item, ou o mesmo pode acontecer quando a baixa em estoque é realizada, mas o processamento da compra falha. O Pessimistic Lock é baseado na ideia de serializar toda a referência a aquele produto em uma única transação, porém em um ambiente de alta demanda, isso se torna um gargalo. A Reserva de Estoque é o processo de separar as quantidades dos itens que foram

selecionados para o carrinho de algum cliente em um status diferente, não disponível a novas compras durante um intervalo de tempo. Assim, se a compra não for finalizada ou for cancelada, fazemos o rollback da quantidade para o estoque. Isso evita vender algo que poderia estar no carrinho de outra pessoa. Contudo, há um aumento de complexidade para a implementação de um serviço de controle de reservas. Por fim temos o Distributed Lock, que funciona de forma semelhante ao Pessimistic Lock, porém considerando que processos fora de uma transação de banco de dados precisam ser executados, sendo assim necessário aplicar o lock da operação em diferentes serviços. Essa solução costuma funcionar com Redis para centralizar os locks aplicados, o que se torna um ponto crítico para essa solução, já que, com a adição da dependência do Redis na arquitetura, todo o funcionamento do sistema é comprometido caso ele falhar, sendo assim necessárias políticas de fallback e resiliência.

Para trabalhar em idempotência podemos aplicar uma chave única em cada requisição e adicionar um status de processamento para essa chave no banco de dados com um índice de unicidade, sendo assim possível confiar na constraint do banco de dados. Se a chave ainda não existe, então o sistema irá criá-la com o status “pendente”. Quando este item começar a ser processado, esse status deve ser atualizado para “processando”. Assim, se um novo envio for realizado, o sistema identifica que esse processamento já está em andamento e, quando finalizado, marcará o status como “completo”.

Para testarmos a garantia da solução, poderíamos fazer os seguintes testes:

- um teste de concorrência, disparando múltiplas requisições diferentes e simultâneas;
- um teste de idempotência, enviando duas requisições diferentes mas com a mesma chave;
- um teste de requisições simultâneas também com a mesma chave, para simular um duplo clique;
- um teste de carga.

PERGUNTA:

5. Sobre o checkout assíncrono e a entrega da solução:

- Você publicaria mensagem na fila antes ou depois de gravar o pedido? Como evitaria pedido fantasma e mensagem fantasma? Inclua retry, DLQ, OpenAPI, testes e prompts de IA relevantes.

RESPOSTA:

Como já descrito na resposta da primeira pergunta, junto com as filas, aplicaria o padrão Transactional Outbox Pattern. Dessa forma, o mais indicado seria a criação do pedido no banco de dados e o registro de que esse pedido precisa ser processado na tabela de outbox dentro de uma mesma transaction, assim ficaríamos blindados para a situação de pedido ou mensagem fantasma. Depois do registro no banco de dados, faria a publicação da mensagem na fila. Para garantir que, mesmo com falhas de rede ou qualquer outro tipo de falha no processo, a mensagem não seja perdida, podemos adicionar políticas de retry com exponential backoff, políticas de timeout e circuit breakers para evitar lidar com serviços já deteriorados, e DLQs para analisar processar mensagens perdidas.

Como uma referência de API para este processo, podemos mapear a requisição principal, que seria um POST na rota /checkout, com o corpo da requisição contendo Id e

quantidade dos produtos, Id do cliente, chave de idempotência e chave de correlação. Este endpoint deve retornar status 202 Accepted e o ID do pedido.

Podemos definir também a rota de acompanhamento do status de processamento deste pedido, já que todo ele será feito de forma assíncrona e tanto o cliente quanto a equipe interna podem precisar acompanhá-lo. Dessa forma, temos um GET na rota `/orders/{orderId}/status`, onde com a requisição devemos enviar a autenticação, e o parâmetro do Id do pedido que já será enviado pelo path da rota. Devemos também validar se quem está fazendo essa solicitação é o cliente ou a equipe interna. Se for o cliente ele só poderá visualizar os próprios pedidos. A resposta deve retornar status 200 OK com o status do pedido.

Alguns testes automatizados interessantes de realizar nesse processo são:

- Conferir se a mensagem é publicada na fila apenas após a transação de Outbox.
- Se a fila estiver fora do ar e a transação de Outbox for publicada, quando a fila voltar a funcionar, ela irá processar as mensagens pendentes.
- Se falhas no sistema estão gerando retries corretamente.
- Se erros sem recuperação estão sendo direcionados para as DLQs.

Para auxiliar nas decisões de arquitetura, foram usados prompts no Claude. A fim de fazer uma contextualização do problema, foi enviado o seguinte trecho:

Considere um sistema de e-commerce que está passando por um grande crescimento e enfrentando problemas de arquitetura. Para consultar o catálogo de produtos, hoje é necessário buscar esses dados de um ERP. Esse processo é realizado de forma síncrona via API, o que está se tornando um gargalo para a plataforma. Além disso, estão lidando com problemas de overselling e lentidão no faturamento. Para corrigir essas questões, precisamos começar a pensar em desacoplamento e processamento assíncrono, priorizando consistência e disponibilidade.

A partir de então, para realizar as decisões de arquitetura, tirando dúvidas e fazendo comparações, os seguintes prompts foram utilizados:

- “Compare as soluções de implementação de cache-aside e refresh-ahead para gerenciamento de caching no catálogo de produtos, com a solução de utilização de um banco de dados próprio realizando a sincronização com o banco de dados do ERP através de uma fila. Além disso, a fim de resolver este problema de forma rápida antes de investir tempo em uma solução mais complexa e mais robusta, quais seriam os impactos negativos de utilizar um simples cache-aside com um TTL pré-fixado e sem estratégia de invalidação?”
- “Quais outras estratégias posso utilizar para resolver o problema de overselling, diferentes da estratégia de reserva de estoque e distributed lock? Além disso, me ajude a prever os modos de falha de cada uma das estratégias.”
- “Como alternativa ao padrão transactional outbox para resiliência no processo de faturamento, enxergo apenas uma solução baseada em polling na tabela de pedidos. Você enxerga uma outra opção?”

- “Na solução de polling na tabela de pedidos, exige que uma requisição seja feita frequentemente na tabela de pedidos no banco de dados. Essa requisição repetitiva pode gerar um aumento significativo no consumo de recursos da máquina, como banda ou CPU?”
- “Quais métricas e índices seriam interessantes para monitorar o sistema de estoque a fim de prever, evitar e, se necessário, atuar com velocidade na correção de overselling?”

Por fim, foi solicitado ao Claude para gerar um contrato formal no padrão OpenAPI a partir da descrição acima dos endpoints. Seu resultado foi validado através do viewer no site <https://www.openapiviewer.com/>. O contrato OpenAPI segue abaixo:

openapi: 3.0.3

info:

title: CaseCellShop - Catálogo e Checkout API

description: >

API responsável por expor o catálogo de produtos com cache, iniciar o processo de checkout assíncrono e permitir a consulta do status de um pedido.

version: 1.0.0

servers:

- url: http://localhost:3000

description: Ambiente local de desenvolvimento

tags:

- name: Products

description: Consulta ao catálogo de produtos

- name: Checkout

description: Início do processo de compra

- name: Orders

description: Acompanhamento do status de pedidos

paths:

/products:

get:

tags: [Products]

summary: Lista o catálogo de produtos

description: >

Retorna os produtos disponíveis, preço e disponibilidade.

A resposta pode ser servida a partir do cache (cache-aside), indicado pelo header X-Cache-Status.

parameters:

- name: page

in: query

required: false

schema:

type: integer

default: 1
- name: pageSize
in: query
required: false
schema:
type: integer
default: 20
responses:
'200':
description: Lista de produtos retornada com sucesso
headers:
X-Cache-Status:
description: Indica se o dado veio do cache ou da fonte
schema:
type: string
enum: [HIT, MISS, STALE]
content:
application/json:
schema:
type: object
properties:
data:
type: array
items:
\$ref: '#/components/schemas/Product'
page:
type: integer
pageSize:
type: integer
total:
type: integer
example:
data:
- id: "prod_001"
name: "Capinha iPhone 15 - Silicone"
price: 49.90
availableStock: 120
updatedAt: "2026-09-18T12:00:00Z"
page: 1
pageSize: 20
total: 1
'500':
\$ref: '#/components/responses/InternalServerError'
'503':
description: Fonte de dados (ERP) e cache indisponíveis, sem fallback possível
content:
application/json:
schema:

\$ref: '#/components/schemas/Error'

/checkout:

post:

tags: [Checkout]

summary: Inicia um checkout assíncrono

description: >

Registra o pedido, reserva o estoque necessário e publica o evento de processamento (Transactional Outbox Pattern).

Retorna imediatamente sem aguardar o faturamento no ERP.

parameters:

- name: Idempotency-Key

in: header

required: true

description: Chave única gerada pelo cliente para evitar duplicidade em retries/duplo

clique

schema:

type: string

format: uuid

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/CheckoutRequest'

example:

customerId: "cust_123"

correlationId: "corr_abc-456"

items:

- productId: "prod_001"

quantity: 2

responses:

'202':

description: Checkout aceito e em processamento assíncrono

content:

application/json:

schema:

\$ref: '#/components/schemas/CheckoutAccepted'

example:

orderId: "order_789"

status: "PENDING"

createdAt: "2026-09-18T12:05:00Z"

'200':

description: >

Retry com a mesma Idempotency-Key de um checkout já processado.

Retorna a mesma resposta da requisição original (idempotência).

content:

application/json:

```

    schema:
      $ref: '#/components/schemas/CheckoutAccepted'
  '400':
    description: Corpo da requisição inválido (ex. quantidade <= 0, produto inexistente)
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
  '409':
    description: Estoque insuficiente para completar a reserva, ou requisição concorrente
    com a mesma Idempotency-Key ainda em processamento
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
        example:
          code: "INSUFFICIENT_STOCK"
          message: "Estoque insuficiente para o produto prod_001"
  '422':
    description: Idempotency-Key ausente ou em formato inválido
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
  '500':
    $ref: '#/components/responses/InternalServerError'

```

/orders/{orderId}/status:

get:

tags: [Orders]

summary: Consulta o status de processamento de um pedido

security:

- bearerAuth: []

parameters:

- name: orderId

in: path

required: true

schema:

type: string

responses:

'200':

description: Status do pedido retornado com sucesso

content:

application/json:

schema:

\$ref: '#/components/schemas/OrderStatus'

example:

orderId: "order_789"

```
    status: "CONFIRMED"
    attempts: 1
    createdAt: "2026-09-18T12:05:00Z"
    updatedAt: "2026-09-18T12:05:04Z"
  '401':
    description: Token de autenticação ausente ou inválido
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
  '403':
    description: >
      O solicitante autenticado não é o dono do pedido nem
      pertence à equipe interna
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
  '404':
    description: Pedido não encontrado
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
  '500':
    $ref: '#/components/responses/InternalServerError'
```

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: >
        O identificador do solicitante (cliente ou equipe interna) é
        extraído das claims do token, nunca enviado como campo solto
        pelo cliente.
```

```
responses:
  InternalError:
    description: Erro interno não esperado
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
```

```
schemas:
  Product:
```

type: object
properties:
 id:
 type: string
 name:
 type: string
 price:
 type: number
 format: float
 availableStock:
 type: integer
 updatedAt:
 type: string
 format: date-time

CheckoutRequest:
type: object
required: [customerId, items]
properties:
 customerId:
 type: string
 correlationId:
 type: string
 description: Id de correlação para rastreabilidade ponta a ponta (opcional; gerado pelo servidor se ausente)
 items:
 type: array
 minItems: 1
 items:
 type: object
 required: [productId, quantity]
 properties:
 productId:
 type: string
 quantity:
 type: integer
 minimum: 1

CheckoutAccepted:
type: object
properties:
 orderId:
 type: string
 status:
 type: string
 enum: [PENDING]
 createdAt:
 type: string

format: date-time

OrderStatus:

type: object

properties:

orderId:

type: string

status:

type: string

enum: [PENDING, PROCESSING, CONFIRMED, FAILED, FAILED_PERMANENT]

attempts:

type: integer

lastError:

type: string

nullable: true

createdAt:

type: string

format: date-time

updatedAt:

type: string

format: date-time

Error:

type: object

required: [code, message]

properties:

code:

type: string

message:

type: string

correlationId:

type: string

nullable: true